

Chiffrements mono-alphabétiques

3 septembre 2026 – B. COLOMBEL

Le compte-rendu de ce TP sera fait sous la forme d'un notebook Jupyter. Il sera à déposer sous le nom

Nom_Prenom-TP1.ipynb

dans le dossier relatif à ce TP dans AMETICE.

Sommaire

1	Rappels d'arithmétique	1
2	Codage / Décodage	3
3	Substitution	4
4	Analyse des fréquences	6

Objectifs :

- Revoir les notions de base d'arithmétique ;
- revoir et se ré-appropriier le calcul modulaire ;
- découvrir les différentes étapes d'un chiffrement ;
- programmer un chiffre historique simple ;
- étudier une méthode de cryptanalyse.

1 Rappels d'arithmétique

L'arithmétique est l'étude mathématique des nombres entiers. Si l'addition, la soustraction et la multiplication des entiers ne posent pas de problème, ce n'est pas le cas pour la division dont le résultat n'est pas toujours un entier. En arithmétique, on utilise la *division euclidienne*.

Théorème 1 (division euclidienne)

Soit a et b deux entiers naturels avec $b \neq 0$. Alors il existe un unique couple $(q; r)$ d'entiers naturels tels que :

$$a = bq + r \text{ avec } 0 \leq r < b$$

On dit que a est le *dividende*, b le *diviseur*, q le *quotient* et r le *reste* dans la division euclidienne de a par b .

Division euclidienne Avec *sagemath*, on obtient le quotient avec l'opérateur `//` et le reste avec `%`.

[1]:

```
a, b = 27, 5
a // b, a % b
```

[1]:

```
(5, 2)
```

Définition 1

Un nombre entier $p \geq 2$ est dit *premier* si il possède exactement deux diviseurs dans $\mathbb{N}$:

1 et lui même

Théorème 2

Soit n un entier naturel avec $n \geq 2$. Alors :

1. n admet au moins un diviseur premier : son plus petit diviseur dans $\mathbb{N}$ autre que 1.
2. Si n est composé, il admet un diviseur premier p tel que $2 \leq p \leq \sqrt{n}$.

Théorème 3

Soit n un entier naturel supérieur ou égal à 2. Alors :

1. n se décompose en un produit de facteurs premiers.
2. Cette décomposition est unique à l'ordre des facteurs près.

[2] :

```
factor(254870)
```

[2] :

```
2 * 5 * 7 * 11 * 331
```

Définition 2

1. le plus grand diviseur commun de a et de b se note $\text{pgcd}(a,b)$;
2. Deux entiers a et b sont *premiers entre eux* si, et seulement si, leurs diviseurs communs sont -1 et 1 .

Cela signifie que $\text{pgcd}(a,b) = 1$.

Le pgcd se détermine grâce à l'algorithme d'Euclide ou plus simplement :

[3] :

```
gcd(85, 130)
```

[3] :

```
5
```

Congruence La relation de *congruence* occupe une place essentielle en arithmétique et importante en informatique.

Définition 3

Soit $n \in \mathbb{N}$ et $(a,b) \in \mathbb{Z}^2$. On dit que a est *congru* à b *modulo* n si a et b ont le même reste dans la division euclidienne par n .

On note :

$$a \equiv b \pmod{n} \text{ ou } a \equiv b \pmod{n}$$

Remarque. Cela signifie que $b - a$ est un multiple de n .

Propriété. Soient $n \in \mathbb{N}$ ($n \geq 2$) et $a \in \mathbb{Z}$.

$$\exists! r \in \{0,1,2,\dots,n-1\}, \quad r \equiv a \pmod{n}$$

Il s'agit du reste de la division euclidienne de a par n .

L'addition et la multiplication sont compatibles avec la congruence modulo n :

Théorème 4

Si $a_1 \equiv a_2 \pmod{n}$ et $b_1 \equiv b_2 \pmod{n}$ alors :

$$a_1 + b_1 \equiv a_2 + b_2 \pmod{n} \quad (1)$$

$$a_1 b_1 \equiv a_2 b_2 \pmod{n} \quad (2)$$

$$a \equiv b \pmod{n} \implies a^p \equiv b^p \pmod{n} \quad (3)$$

[4] :

```
(18*11-5)%15
```

[4]: 13

[5]: 18%15 * 11%15 - 5%15

[5]: -2

[6]: 13 % 15 == -2%15

[6]: True

Définition 4 (Inverse modulaire)

L'entier x est *inversible modulo* m si :

$$\exists x' \in \mathbb{Z}, \quad xx' \equiv 1 \pmod{m}$$

Théorème 5

L'entier x est inversible modulo m si, et seulement si :

$$\text{pgcd}(x, m) = 1$$

Nous verrons dans la suite du cours comment déterminer, s'il existe, l'inverse modulaire d'un nombre.

2 Codage / Décodage

Pour simplifier, on ne travaille pour l'instant qu'avec des messages en majuscules sans espaces.

Le principe général d'un chiffrement est le suivant :

- On numérote les 26 lettres de l'alphabet de 0 (pour A) à 25 (pour Z) ; c'est l'opération de codage ;
- On calcule les images de ces 26 entiers par une fonction appelée *fonction de chiffrement* ;
- On prend les restes des images précédentes dans la division euclidienne par 26, c'est-à-dire que l'on travaille dans $\mathbb{Z}/_{26\mathbb{Z}}$.
- On remplace les lettres composants le message initial par celles qui correspondent aux restes.

Les première et dernière étapes consistent à faire correspondre lettres et nombres suivant le tableau ci-contre.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

L'histoire de l'informatique nous donne une manière naturelle de procéder grâce au code ASCII :

- L'American Standard Code for Information Interchange (Code américain normalisé pour l'échange d'information), plus connu sous l'acronyme ASCII ([aski :]) est une norme de codage de caractères en informatique ancienne et connue pour son influence incontournable sur les codages de caractères qui lui ont succédé. Elle était la plus largement compatible pour ce qui est des caractères latins non accentués.
- ASCII contient les caractères nécessaires pour écrire en anglais. Les caractères accentués sont fournis par d'autres normes, comme aujourd'hui, l'Unicode. Le jeu de caractères codés ASCII est le principal système qui a permis l'échange de textes en anglais à un niveau mondial, limitant ainsi l'usage des langues locales au travers d'extensions régionales.

Sources : Wikipédia

La table 1 page 4 représente une partie de la table ascii.

Par exemple, : vaut 58, @ vaut 64, A vaut 65, B vaut 66, a vaut 97, etc.

Caractères		!	"	#	\$	%	&	'	(	)	*	+	,	-	.
Code	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46
Caractères	/	0	1	2	3	4	5	6	7	8	9	:	;	<	=
Code	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61
Caractères	>	?	@	A	B	C	D	E	F	G	H	I	J	K	L
Code	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76
Caractères	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	[
Code	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91
Caractères	\	]	^	_	'	a	b	c	d	e	f	g	h	i	j
Code	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106
Caractères	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y
Code	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121
Caractères	z	{		}	~										
Code	122	123	124	125	126										

TABLE 1 – Éléments de la table ASCII

Avec *sagemath*, on peut passer d'un caractère à son code ascii et vice versa avec les fonctions `ord()` et `chr()`.

[7]: `ord('A')`

[7]: 65

[8]: `chr(65)`

[8]: 'A'

Exercice 1.

1. Que fait la fonction suivante? Recopier ce code et tester avec `s = 'ABCYZ'`.

[9]:

```
def code(s) :
    tab = []
    for i in s :
        tab.append(ord(i)-65)
    return tab
```

2. Écrire une fonction `decode(tab)` qui permet de faire le chemin inverse.

3 Substitution

Le *chiffrement par substitution* consiste à substituer dans un message chacune des lettres de l'alphabet par une autre (du même alphabet ou éventuellement d'un autre alphabet).

Le chiffrement par substitution est une technique de chiffrement utilisée depuis bien longtemps puisque le chiffre de César en est un cas particulier.

Chiffre de César. Le texte chiffré s'obtient en remplaçant chaque lettre du texte clair original par une lettre à distance fixe, toujours du même côté, dans l'ordre de l'alphabet. Pour les dernières lettres (dans le cas d'un décalage à droite), on reprend au début. Par exemple avec un décalage de 3 vers la droite, A est remplacé par D, B devient E, et ainsi jusqu'à W qui devient Z, puis X devient A etc. Il s'agit d'une permutation circulaire de l'alphabet.

Chiffrer avec César consiste à choisir un nombre appelé **clé** pour transformer une lettre de l'alphabet et une autre lettre par le jeu des transformations suivantes :

$$\begin{array}{ccccccc}
 \text{Alphabet} & \longrightarrow & \{0; 1; \dots; 25\} & \longrightarrow & \mathbb{Z}_{26\mathbb{Z}} & \longrightarrow & \text{Alphabet} \\
 \text{lettre} & \longmapsto & \text{nombre} & \xrightarrow{f} & \text{nombre} & \longmapsto & \text{lettre} \\
 M & \longmapsto & n & \longmapsto & n_c \equiv n + \text{clé} \pmod{26} & \longmapsto & M_c
 \end{array}$$

Exercice 2. Nous voulons programmer une fonction `cesar(chaine, n)` qui prendra comme paramètre une chaîne de caractères `chaine` en majuscule sans espace et un nombre n représentant le décalage.

1. Écrire cette fonction `cesar(chaine, n)` pour obtenir en sortie la chaîne chiffrée. N'oubliez pas de la tester.
2. Proposer une méthode pour déchiffrer le texte. La tester.

Remarque. Le chiffre de César est très peu sûr puisqu'il n'y a que 26 clés possibles (y compris la clé nulle, qui ne modifie pas le texte).

Chiffrement affine Un *chiffrement affine* consiste à se donner deux entiers naturels a et b , que l'on pourrait appeler les *clés de chiffrements*, puis effectuer les transformations suivantes sur chacune des lettres que l'on veut chiffrer :

$$\begin{array}{ccccccc}
 \text{Alphabet} & \longrightarrow & \{0; 1; \dots; 25\} & \longrightarrow & \mathbb{Z}_{26\mathbb{Z}} & \longrightarrow & \text{Alphabet} \\
 \text{lettre} & \longmapsto & \text{nombre} & \xrightarrow{f} & \text{nombre} & \longmapsto & \text{lettre} \\
 M & \longmapsto & n & \longmapsto & n_c \equiv (an + b) \pmod{26} & \longmapsto & M_c
 \end{array}$$

où f est la fonction affine définie par $f(n) = an + b$.

Remarques.

1. Si $a = 1$, on retombe sur un chiffre de César.
2. Pour qu'un chiffrement soit cohérent, il doit pouvoir être déchiffré (en connaissance de la clé) sans ambiguïté. Ainsi, deux lettres différentes ne peuvent être chiffrées par une même lettre. Ainsi, deux nombres correspondants à deux lettres différentes ne peuvent avoir la même image par la fonction affine $f(n) = an + b$, c'est à dire que la fonction de chiffrement doit être *injective*.
C'est le cas si a et 26 sont premiers entre eux.

Exercice 3. Écrire une fonction `chiffre_affine(mot_a_chiffrer, a, b)` qui permet de chiffrer le mot `mot_a_chiffrer` par la fonction affine $f(n) = an + b$.

Déchiffrement affine. On dispose d'un mot chiffré par un chiffrement affine de clefs a et b , ie, par la fonction affine $f(n) = an + b$.

Soit M_c une lettre du mot chiffré correspondant au nombre n_c . Elle provient du chiffrement de la lettre M correspondant à l'entier n :

$$\begin{array}{ccc}
 \{0; 1; 2; \dots; 25\} & \longrightarrow & \{0; 1; 2; \dots; 25\} \\
 n & \xrightarrow{f} & n_c = an + b \pmod{26}
 \end{array}$$

Il s'agit alors d'exprimer n en fonction de n_c .

$$\begin{aligned}
 n_c \equiv an + b \pmod{26} &\iff an \equiv n_c - b \pmod{26} \\
 &\iff a^{-1} \times a \times n \equiv a^{-1} \times (n_c - b) \pmod{26} \quad \text{si } a \text{ est inversible modulo } 26 \\
 &\iff n \equiv a^{-1}(n_c - b) \pmod{26}
 \end{aligned}$$

Ainsi, à la condition que a soit inversible dans $\mathbb{Z}_{26\mathbb{Z}}$, on a :

$$\boxed{n \equiv a^{-1}(n_c - b) \pmod{26}}$$

Exercice 4.

1. Écrire une fonction `inv(a, p=26)` qui,
— vérifie si a est inversible modulo p (pgcd) ;

- détermine, s'il existe, l'inverse modulaire de a modulo p (force brute admise)
 - retourne l'inverse de a modulo p (ou -1 si a n'est pas inversible).
2. Écrire une fonction `mot_dechiffre = dechiffre_affine(mot_a_dechiffrer, a, b)` qui permet de déchiffrer le mot saisi en paramètre `mot_a_dechiffrer` avec les clés a et b qui ont été utilisé dans le chiffrement affine.

Remarque. Un chiffrement affine est très peu sur. En effet, il n'y a que $26^2 = 676$ clés différentes possibles. Il est très facile, avec un ordinateur, de toutes les tester.

Chiffre par substitution. Voici un exemple de chiffre par substitution :

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	Z	E	R	T	Y	U	I	O	P	Q	S	D	F	G	H	J	K	L	M	W	X	C	V	B	N

Il ne s'agit pas ici d'un chiffrement affine.

L'ordre de l'alphabet latin étant connu, il suffit de donner la suite des 26 signes correspondant, qui est la clef de chiffrement. L'alphabet du chiffré peut être le même que celui du clair (en changer n'introduit pas de sécurité supplémentaire). Pour l'alphabet latin, ceci permet de construire

$$26! = 403\,291\,461\,126\,605\,635\,584\,000\,000$$

substitutions différentes soit le nombre de permutations d'un ensemble à 26 éléments.

Il n'est donc plus question de faire une recherche exhaustive pour casser un chiffre par substitution. Heureusement, les linguistes (et les statisticiens) sont passés par là.

4 Analyse des fréquences

Un *chiffrement par substitution* consiste à transformer une lettre en une seule autre lettre en changeant l'ordre des 26 lettres de l'alphabet. Ainsi, chacune des lettres du message à chiffrer est toujours remplacée par la même lettre dans le message chiffré.

Il existe une méthode quasi-automatisée pour « casser » un chiffre mono-alphabétique. Cette méthode s'appuie sur les fréquences d'apparition des lettres dans la langue française.

Le lien suivant Wikipedia l'explique en détail. https://fr.wikipedia.org/wiki/Analyse_fréquentielle

Exercice 5.

- Proposer une méthode pour donner les fréquence d'apparition des lettres d'un texte.
- Écrire une fonction `StatTexte(str)`. Cette fonction renverra les résultats sous la forme d'un tableau présentant les pourcentages par lettre.
Par exemple, `[2.1, 1.5, ...]` donne lettre A 2.1 % d'apparition.
- Déchiffrer le texte du fichier `texte_chiffre`.